

VENTURE / PRODUCT / LAUNCH REPORT

WORKS

Autonomous Software Execution Infrastructure

Professional venture report • Product Spec V1 • Launch blueprint • 31 August 2026

Code is an input. Verified software state is the output.

Venture thesis	Initial wedge	Long-term category
Replace machine-centric CI with a durable execution control plane.	Drop-in GitHub-compatible execution with better speed, cache, economics and observability.	Execution substrate for autonomous software engineering.

CONFIDENTIAL WORKING DOCUMENT

1. Executive Summary

WORKS is a standalone developer-infrastructure venture. It is designed to execute, schedule, recover, cache, verify and evidence software work across arbitrary repositories and compute environments. GitHub, GitLab and other code hosts are integrations, not architectural owners of execution.

The near-term commercial wedge is intentionally conservative: let teams keep their existing GitHub Actions workflows while moving execution onto WORKS workers. The larger product then replaces static machine-centric pipelines with a durable Work Graph that can schedule deterministic tasks, compute workers and eventually software agents as first-class execution resources.

The opportunity is real, but the market is not empty. Buildkite already separates a hosted control plane from self-hosted agents and supports dynamic pipelines. Nx Cloud already distributes task graphs, uses affected execution, remote caching, flaky-task recovery and AI self-healing. Depot offers drop-in GitHub runners and high-performance cache. Therefore WORKS cannot win by claiming 'faster runners', 'remote cache', 'BYOC', or 'AI CI' as unique features. Those are table stakes.

Investment / build thesis

- Wedge: make adoption almost frictionless through GitHub Actions compatibility.
- Core primitive: Work, a durable objective plus execution graph, policy, state, artifacts and evidence.
- Differentiator: objective- and evidence-driven execution rather than static workflow-to-machine assignment.
- Moat candidate: cross-work execution data that improves scheduling, reuse, failure recovery and verified remediation.
- Endgame: a vendor-neutral execution substrate for both human-authored CI and autonomous software agents.

V1 success definition

Metric	V1 target	Why it matters
Time to first successful Work	< 5 min	Adoption friction
GitHub migration median	< 15 min	Commercial wedge
P50 time-to-green improvement	>= 50% on qualified workloads	Customer ROI
Redundant compute eliminated	>= 50% on repeatable workloads	Cost advantage
Recoverable infra failures auto-recovered	>= 80%	Reliability
Control-plane availability	> 99.9% target	Trust

2. Problem Definition

2.1 CI is still machine-centric

Traditional CI systems ask teams to encode jobs, machines, matrices, dependencies, caches, retries and conditions in advance. This is operationally understandable, but it makes the machine and pipeline configuration the primary abstraction rather than the verified outcome.

The WORKS thesis is that the higher-value abstraction is: objective + constraints + evidence requirement → minimum safe execution graph → verified result.

2.2 Redundant computation is structurally expensive

Modern repositories repeatedly reinstall dependencies, rebuild unchanged targets, rerun unaffected tests and recreate equivalent artifacts. Existing competitors prove the demand for remote caching and affected execution. WORKS must therefore treat computation reuse as a native execution primitive, not a bolt-on cache.

2.3 Failure recovery wastes developer attention

A runner disappearing, a transient network timeout, an out-of-memory condition and a genuine code regression are materially different failures. Most CI surfaces still converge on the same primitive: red job, logs, human attention. WORKS should classify failures and automatically perform safe remediation before escalating.

2.4 Agentic development multiplies execution volume

Software agents can generate many more build-test-review-repair cycles than human developers. That shifts CI from a human-triggered utility into machine-to-machine infrastructure. The economic bottleneck becomes verified work throughput: how cheaply and reliably can an organization convert intents and changes into trusted outcomes?

2.5 Fragmentation

- Repository host owns triggers and status.
- CI provider owns workflow execution.
- Cloud provider owns machines.
- Build systems own dependency graphs and caches.
- Deployment systems own releases.
- Coding agents create additional execution loops.
- Security and audit evidence is scattered across all of them.

WORKS should not rebuild every layer. It should own the execution contract that coordinates them.

3. Market Reality & Competitive Landscape

The market confirms the pain but also raises the bar. As of August 2026, several competitors already cover substantial portions of the proposed feature set.

Competitor	Current strength	Threat to WORKS	Strategic response
GitHub Actions	Distribution, ecosystem, native repo integration	Default incumbent	Compatibility first; do not require repo migration
Buildkite	Control plane + self-hosted agents, dynamic pipelines	Closest architecture analogue	Differentiate at Work/evidence/objective layer
Nx Cloud	Task graph, affected, distribution, caching, self-healing	Strongest execution-intelligence analogue	Remain framework/runtime neutral and broader than monorepo CI
Depot	Drop-in runners, fast cache, per-second usage	Strong wedge competitor	Match low-friction migration; compete above runner layer
Harness	Enterprise CI/CD, governance, test intelligence	Enterprise platform pressure	Narrow initial scope; superior execution model
Dagger	Portable programmable CI	Developer portability	Treat portability as required, not sufficient

3.1 Verified competitive facts

GitHub currently publishes hosted-runner pricing including \$0.006/min for a standard 2-core x64 Linux runner and \$0.062/min for standard macOS, with partial job minutes rounded up to whole minutes. This makes execution economics visible enough for challengers to compete directly on cost.

Buildkite's current Pro plan lists dynamic pipelines, up to 250 concurrent self-hosted agents, and hosted Linux execution billed per vCPU minute. This validates the control-plane + execution-agent model.

Nx Cloud explicitly positions itself as task-based CI rather than traditional VM-based CI. It advertises remote caching, affected execution, distributed agents, flaky-task detection and self-healing CI. Its documentation says tasks are allocated using dependencies and historical runtimes, and agents can scale based on PR size.

Depot markets its GitHub Actions runners as a drop-in replacement, with a one-line runs-on change, per-second usage accounting, distributed cache, and plans that include Linux, Windows and macOS runners. That is a strong benchmark for WORKS onboarding.

3.2 What is NOT a moat

- Faster virtual machines.
- Remote cache.
- Self-hosted or BYOC workers.
- Dynamic pipelines.
- Affected-test execution.
- AI log summaries.
- Automatic retry of flaky tasks.
- A pretty CI dashboard.

A credible venture must assume competitors can copy individual features. The defensible unit has to be the execution model, accumulated execution/evidence graph and integration depth.

4. Product Definition

4.1 Category

Initial category: CI/CD and developer infrastructure. Intended category creation: Autonomous Software Execution Infrastructure.

4.2 Product promise

Connect a repository or submit an objective. WORKS determines or accepts the required execution graph, schedules it across compatible workers, reuses valid prior computation, recovers from safe infrastructure failures, produces artifacts, and records structured evidence of the result.

4.3 Core primitive: Work

Field	Purpose
source	Commit, PR, webhook, CLI, API, agent or scheduled trigger
objective	What outcome is requested
graph	Execution DAG and dynamic additions
requirements	OS, architecture, resources, deadline, cost, confidence
policy	Permissions, approvals, secret access, deployment authority
state	Durable lifecycle independent of any worker
executions	Attempts, leases, timings and outcomes
artifacts	Build outputs and deliverables
evidence	Tests, scans, builds, signatures and deployment proof
approvals	Human or policy gates when authority requires them

4.4 State model

CREATED → PLANNING → QUEUED → RUNNING → VERIFYING → SUCCEEDED, with explicit BLOCKED, FAILED and CANCELLED states. Worker state is subordinate to Work state. Losing a worker must not mean losing the Work.

4.5 Native Work vs compatibility mode

Mode	Purpose	Customer value
Compatibility	Run existing GitHub Actions on WORKS-compatible workers	Immediate adoption without pipeline rewrite
Native Work	Use WORKS DAG, cache, evidence, scheduling and policy directly	Maximum performance and intelligence
Objective mode (later)	Submit desired outcome + constraints	Autonomous execution planning

5. V1 Functional Requirements

5.1 Control Plane

- Multi-tenant organization, repository and project model.
- Work API with idempotent creation and durable state.
- DAG persistence and dependency resolution.
- Queue, leases, heartbeat and lost-worker recovery.
- Scheduler with hard capability constraints and soft optimization.
- Artifact metadata and content-addressed storage references.
- Structured evidence store.
- Audit event stream.
- Usage and cost metering.

5.2 Worker Runtime

- Linux first; Windows and macOS follow in V1 program.
- Outbound secure connection to control plane.
- Worker identity and capability advertisement.
- Ephemeral Work credentials.
- Sandboxed process/container execution.
- Heartbeat, lease renewal, cancellation and graceful shutdown.
- Local cache with safe eviction.
- Artifact upload/download.
- Log streaming with secret masking.

5.3 Scheduler

Hard constraints must always dominate optimization. Eligible workers are then scored using availability, cache locality, expected duration, queue pressure, region, trust and cost. V1 can begin with deterministic heuristics; historical prediction can improve after sufficient telemetry exists.

5.4 Content-addressed execution

Deterministic nodes receive a fingerprint derived from declared inputs, source state, command, environment and toolchain. A cache hit is valid only when the platform can prove equivalent inputs and policy allows reuse.

5.5 Failure handling

Class	Default V1 response
Worker lost	Requeue node on compatible worker
Network/transient	Bounded retry with backoff
OOM	Retry on larger eligible worker if policy allows
Known flaky test	Isolated bounded retry
Cache corruption	Invalidate affected entry and re-execute
Code/test failure	Fail node and preserve evidence
Unknown	Escalate with complete diagnostic bundle

6. Security & Trust Model

CI executes untrusted or semi-trusted code with access to source, credentials and deployment paths. Security is therefore a product requirement, not enterprise garnish applied six months before procurement.

6.1 V1 security baseline

- TLS for all control-plane/worker communication.
- Unique worker identity and revocation.
- Short-lived, Work-scoped credentials.
- Encrypted secret storage and masking.
- No secrets included in cache keys or reusable artifacts in plaintext.
- Tenant-isolated artifact and cache namespaces.
- RBAC for organization, project, worker pools and sensitive operations.
- Immutable audit events for Work lifecycle and privileged actions.
- Artifact/evidence integrity hashes and signing path.
- Explicit trust classes for customer-owned, hosted and privileged workers.

6.2 Threats that must be designed for

- Malicious pull request exfiltrating secrets.
- Compromised worker impersonation.
- Cross-tenant cache poisoning.
- Artifact substitution.
- Replay of expired execution credentials.
- Log-based secret leakage.
- Untrusted forks receiving privileged Work authority.
- Supply-chain compromise of worker binaries or update channel.

6.3 Enterprise readiness path

SSO/SAML, SCIM, customer-managed keys, private networking, policy-as-code, retention controls, SIEM export, dedicated control planes, regional residency and formal compliance programs belong on the enterprise roadmap after the security architecture supports them cleanly.

7. User Experience

7.1 Five-minute onboarding

The ideal path is: install CLI → authenticate → connect repository → inspect detected project → run first Work. Configuration is generated but never hidden. A team can progressively move from compatibility mode to native Work graphs.

7.2 CLI surface

Command	Purpose
works init	Detect repository and create project configuration
works run	Execute local/native Work
works status	Inspect Work and worker state
works logs	Stream or retrieve execution logs
works worker install	Register a worker
works cache	Inspect/invalidate cache
works doctor	Validate local/worker environment

7.3 Web control surface

The default interface is execution-first and exception-first. The first screen answers: what is running, what is blocked, what failed, what needs authority, what did we save, and what changed relative to baseline.

- Live Work graph with critical path.
- Queue and worker-pool health.
- Failure classification and recovery actions.
- Cache hit/miss and avoided compute.
- Artifacts and evidence.
- Cost and runtime baseline comparison.
- Audit history.
- Repository and worker configuration.

7.4 Developer ergonomics

Every failure should expose a stable Work ID, node ID, fingerprint, worker identity, retry history and diagnostic bundle. Copy-pasting 4,000 lines of colored terminal output into chat should not be the observability strategy. Civilization can aim slightly higher.

8. Architecture Blueprint

Recommended V1 architecture: a cloud control plane with stateless API services, durable relational state, queue/event infrastructure, object storage/CAS, and independently deployable worker runtime.

Component	Recommended responsibility
API Gateway	Authentication, rate limits, public API
Work Service	Lifecycle, idempotency, state transitions
Graph Engine	DAG validation, ready-node calculation
Scheduler	Eligibility, scoring, assignment, leases
Worker Gateway	Persistent worker sessions, heartbeat, commands
Artifact/CAS	Content-addressed artifacts and cache objects
Evidence Service	Structured verification records
Audit/Metering	Immutable events, usage, cost attribution
Web App	Operations, diagnostics, configuration

8.1 Suggested implementation stack

A pragmatic starting stack is Go for control-plane and worker services, PostgreSQL for durable metadata, object storage compatible with S3 for artifacts/CAS, and a queue/event layer selected after load testing.

TypeScript/React can power the web product. gRPC or a compact streaming protocol can serve worker-control traffic; the public API should remain easy to consume over HTTP.

8.2 Architectural rules

- Control plane is source of truth; workers are disposable.
- Worker protocol is versioned and runtime-independent.
- Every state mutation is idempotent or guarded by version/lease semantics.
- Cache correctness beats cache hit rate.
- Evidence must be attributable to exact source + environment + execution.
- Scheduling decisions are explainable and observable.
- No V1 service requires AI to remain operational.

9. Commercial Strategy

9.1 Ideal Customer Profile

Initial ICP: engineering organizations with roughly 10–200 developers, GitHub-centric workflows, meaningful CI spend, pipelines commonly above 10 minutes, monorepos or large repositories, Docker-heavy workloads, parallel development and early adoption of coding agents.

9.2 Buyer and user

Role	Pain	Message
Developer	Slow feedback, flaky runs	Get to green faster without babysitting CI
Platform/DevOps	Runner operations and fragmented tooling	One execution control plane across compute
Engineering leader	Developer time + infrastructure cost	Measure cost per verified change
Security	Uncontrolled credentials and evidence gaps	Scoped execution authority and auditability

9.3 Land-and-expand motion

1. Land with drop-in GitHub-compatible workers.
2. Expose runtime, cache and cost savings immediately.
3. Enable native cache and artifact graph.
4. Move expensive pipelines to native Work DAGs.
5. Add private/BYOC worker pools and governance.
6. Introduce evidence-driven verification and autonomous remediation.
7. Expand into agent execution.

9.4 Pricing hypothesis

Tier	Hypothesis	Purpose
Developer	Free/BYOC + limited cloud usage	Adoption and OSS
Pro	~\$20/user/mo + compute	Small professional teams
Team	~\$40/user/mo + compute	Shared cache, analytics, scheduling
Enterprise	Custom	Private infra, governance, SLA, compliance

These are hypotheses, not facts. Pricing must be validated through interviews, pilot economics and competitor switching analysis before public launch.

10. Unit Economics & Value Model

10.1 North-star economic metric

Verified Work Throughput = verified outcomes / (wall time + compute cost + required human attention). This avoids the trap of optimizing runner speed while ignoring retries, idle capacity, manual diagnosis and unnecessary execution.

10.2 Customer ROI model

For each repository, WORKS should calculate a baseline and continuously attribute savings:

Dimension	Measurement
Wall time	Baseline critical-path runtime vs WORKS
Compute	CPU/GPU minutes executed vs avoided
Cache	Reused node runtime and transferred bytes
Reliability	Retries avoided, infra failures recovered
Human attention	Failures auto-resolved vs manually escalated
Cost	Provider/BYOC cost vs baseline estimate

10.3 Commercial proof

A pilot should not be declared successful because a demo looked fast. It succeeds when a real team can show lower time-to-green and/or lower cost per verified change without reducing verification confidence.

11. Go-to-Market Launch Plan

Phase 0 - Founder validation (Weeks 0-3)

- Interview 20–30 engineering/platform leaders with high CI usage.
- Collect anonymized pipeline structures, runtimes, queue delays, failure categories and monthly spend.
- Test willingness to switch runners without changing workflows.
- Identify 3–5 design partners willing to run shadow/pilot workloads.
- Reject segments whose problem is merely 'our tests are slow code'.

Phase 1 - Technical alpha (Weeks 2-8)

- Linux worker runtime.
- Durable Work state and DAG.
- Scheduler, leases, heartbeat and recovery.
- Log streaming.
- Artifact/CAS foundation.
- GitHub App/webhook integration.
- Compatibility runner path.
- Internal benchmark suite.

Phase 2 - Design-partner beta (Weeks 8-16)

- Remote cache and execution fingerprinting.
- Cost/runtime analytics.
- Worker pools and BYOC.
- Failure classification.
- Evidence records.
- RBAC/audit baseline.
- Migration tooling and GitHub checks/status.

Phase 3 - Paid beta (Months 4-6)

- Billing/metering.
- Windows/macOS execution where economics support it.
- SLOs and incident process.
- Security review and penetration test.
- Native Work graph authoring.
- First paid customer contracts.

Phase 4 - V1 GA (Months 6-9)

- Stable public API and worker protocol.
- Documented migration and rollback.
- Production support model.
- Usage limits and abuse controls.
- Enterprise discovery pipeline.
- Published benchmark methodology and customer evidence.

12. Engineering Execution Plan

12.1 Initial repository structure

A monorepo is appropriate initially because protocol, CLI, worker, control plane and web UI will evolve together.

Path	Responsibility
/cmd/works	CLI
/cmd/works-worker	Worker daemon
/services/api	Public/control API
/services/scheduler	Scheduling and leases
/services/worker-gateway	Worker sessions
/packages/protocol	Versioned shared contracts
/packages/work-graph	DAG model and validation
/packages/evidence	Evidence schemas
/web	Operations/product UI
/infra	Development/deployment manifests
/docs/adr	Architecture decisions
/benchmarks	Reproducible competitive/performance tests

12.2 First 12 engineering milestones

8. Define Work, Node, Attempt, Worker, Lease, Artifact and Evidence schemas.
9. Implement local single-node Work execution.
10. Persist Work/DAG state in PostgreSQL.
11. Implement worker registration and heartbeat.
12. Implement lease-based scheduling.
13. Recover from worker loss without corrupting Work state.
14. Stream logs and persist terminal result.
15. Add artifact upload/download with hashes.
16. Implement deterministic node fingerprints.
17. Add local + organization cache.
18. Connect GitHub events and commit statuses.
19. Run a real repository through compatibility and native modes.

12.3 Quality gates

- Property/state-machine tests for Work transitions.
- Chaos tests for worker disconnect and duplicate delivery.
- Cache poisoning/correctness tests.
- Load tests for scheduler and log streaming.
- Cross-platform worker conformance suite.
- Security tests for fork/secret boundaries.
- Reproducible benchmark harness.

13. Data Model & APIs

13.1 Core entities

Entity	Key relationships
Organization	Users, repositories, worker pools, billing
Repository	Works, integrations, policies
Work	Graph, attempts, artifacts, evidence, approvals
Node	Dependencies, fingerprint, execution requirements
Attempt	Worker lease, timestamps, logs, exit/result
Worker	Pool, identity, capabilities, health
Artifact	Content hash, provenance, retention
Evidence	Source, type, result, fingerprint, signature
AuditEvent	Actor, action, resource, timestamp

13.2 API examples

POST /v1/works creates idempotent Work. GET /v1/works/{id} returns durable state. POST /v1/workers/register establishes worker identity. Worker streaming channels receive leases and send heartbeat/result events. Artifact APIs use signed short-lived transfer URLs rather than proxying large payloads through application servers.

13.3 Compatibility contract

The worker protocol and public Work schema should be versioned independently. Backward compatibility windows and explicit capability negotiation are required before external workers are encouraged.

14. Observability, SRE & Operations

14.1 Platform SLOs

SLO	Initial target
Control-plane availability	99.9%
Work creation API P95	< 500 ms excluding external auth
Eligible-node scheduling P95	< 1 s under normal load
Lost-worker detection	Configurable, target < 30 s
Audit event durability	No acknowledged event loss

14.2 Required telemetry

- Work/node/attempt state transitions.
- Queue depth and age by pool.
- Worker capacity, utilization and churn.
- Scheduler decision reasons.
- Cache hit rate, bytes, latency and invalidations.
- Artifact transfer latency and errors.
- Failure classes and remediation outcomes.
- Cost per Work/repository/organization.
- Critical-path duration.
- External dependency health.

14.3 Incident design

The system needs degraded modes. If analytics is unavailable, execution continues. If artifact storage is degraded, nodes requiring artifact persistence must fail closed rather than claim success. If scheduler intelligence is unavailable, a deterministic fallback scheduler should operate. If the control-plane state store is unavailable, new execution must stop safely.

15. Risk Register

Risk	Severity	Mitigation
Crowded market	High	Compete above runner layer; prove unique Work/evidence model
Cache correctness bug	Critical	Content-addressing, declared inputs, sandboxing, conservative invalidation
Secret exfiltration	Critical	Scoped credentials, fork policies, isolation, audit
Worker compromise	Critical	Identity, revocation, ephemeral credentials, signed updates
GitHub dependency	High	Provider-neutral core and public API
Overbuilding AI too early	High	Deterministic V1; AI optional and policy-gated
Weak differentiation	High	Measure Work throughput, not feature checklist
Unprofitable hosted compute	High	BYOC + metering + margin guardrails
Enterprise sales drag	Medium	Start developer/platform wedge; enterprise later
Cross-platform complexity	Medium	Linux-first conformance, staged OS expansion
Data residency/compliance	Medium	Architect tenant/region boundaries before certifications
Objective execution unsafe	High	Later phase, explicit authority and evidence gates

16. Validation & Benchmark Program

The '100x GitHub' ambition must be converted into falsifiable benchmarks. The product should never publish a universal 100x performance claim. Instead, define a composite throughput thesis and publish workload-specific evidence.

16.1 Benchmark suite

- Cold build: no cache.
- Warm repeat build.
- Small PR affecting one package.
- Large PR affecting many packages.
- Flaky-test scenario.
- Worker-loss mid-execution.
- OOM requiring reschedule.
- Docker-heavy build.
- High-concurrency burst.
- Agent-generated rapid commit loop.

16.2 Compare against

- GitHub-hosted Actions baseline.
- GitHub self-hosted baseline where appropriate.
- Depot drop-in runner baseline.
- Buildkite agent pipeline for comparable workloads.
- Nx Cloud for monorepo/task-graph workloads.

16.3 Publish

Every benchmark must disclose repository revision, machine shape, region, cache state, concurrency, repetitions, median/P95, cost assumptions and whether numbers are vendor-provided or independently measured.

17. Company Formation & Operating Checklist

Before accepting customer workloads

- Choose legal entity and jurisdiction with professional legal/accounting advice.
- Founder/IP assignment and contributor agreements.
- Trademark/domain screening for final brand; WORKS is a working name.
- Privacy policy, terms, DPA and subprocessors list.
- Security contact and vulnerability disclosure process.
- Incident response owner and on-call procedure.
- Backups and disaster-recovery tests.
- Cyber/security insurance assessment.
- Billing provider and tax/VAT treatment.
- Data retention/deletion policy.
- Open-source license strategy.
- Customer pilot agreement with liability boundaries.

Operating cadence

Cadence	Review
Daily	Reliability, customer blockers, benchmark regressions
Weekly	Product metrics, design partners, burn, security issues
Biweekly	Roadmap and architecture decision review
Monthly	Unit economics, competitive movement, runway
Quarterly	Category thesis, pricing, enterprise readiness

18. Hiring Plan

Do not begin with a large team. The first hires should reduce existential technical and commercial risk.

Priority	Role	Mission
1	Founding infrastructure engineer	Worker runtime, scheduler, distributed systems
2	Founding product/full-stack engineer	CLI, web operations, integrations
3	Developer relations / solutions engineer	Design partners, migration, docs
4	Security engineer or retained specialist	Threat model, isolation, reviews
5	SRE/platform engineer	Reliability as customer load grows

A dedicated ML team is not a V1 requirement. Historical models and AI remediation should arrive only after the platform produces enough high-quality execution data to justify them.

19. First 30 / 60 / 90 Days

Days 0–30: prove the pain

- Complete 20–30 customer interviews.
- Secure at least three serious design partners.
- Freeze Work V1 schema and worker protocol draft.
- Build local worker + durable control-plane skeleton.
- Create benchmark corpus and GitHub baseline.
- Resolve final name/domain/trademark direction.
- Write threat model before secrets/deployment access exists.

Days 31–60: prove execution

- Run real GitHub repositories through WORKS.
- Demonstrate worker-loss recovery.
- Ship artifact/CAS and first cache implementation.
- Expose critical-path and cost telemetry.
- Implement GitHub status/check integration.
- Measure cold/warm/affected benchmarks against baseline.

Days 61–90: prove customer value

- Deploy design-partner pilots.
- Measure time-to-green and compute economics.
- Fix onboarding until first Work is consistently <5 minutes.
- Add BYOC worker pool.
- Complete external security review of core execution path.
- Select paid-beta packaging based on observed value.
- Decide go/no-go for macOS/Windows based on customer demand.

20. Decision Gates

Gate	Pass condition	If it fails
Problem	Design partners report meaningful CI pain/spend	Narrow ICP or stop
Adoption	Migration <15 min median	Improve compatibility before native graph
Performance	Material TTG/cost improvement on real workloads	Do not market speed
Reliability	Worker loss/retry never corrupts state	Block external beta
Security	Threat model + review passes	No privileged workloads
Commercial	At least 2–3 partners willing to pay	Revisit packaging/value
Differentiation	Native Work shows value beyond runner swap	Avoid commodity runner business

Kill criteria

The venture should be reconsidered if customers only value cheaper compute, if compatibility economics cannot beat incumbents without unsustainable subsidy, or if native Work/evidence fails to create measurable additional value. Founder enthusiasm is not product-market fit, however inconvenient that may be.

21. Long-Term Roadmap

Stage	Product evolution
V1	Durable execution, workers, scheduling, cache, GitHub wedge, evidence
V1.5	Cross-repo graphs, predictive scheduling, richer policy, GitLab
V2	Agent workers, dynamic Work Graph, failure intelligence, verified remediation
V3	Objective-driven execution, autonomous verification, multi-agent software work
Platform	Vendor-neutral execution substrate across code, agents, compute and delivery

End-state architecture

Humans and software agents submit Work. The control plane resolves objectives into policy-constrained graphs, schedules deterministic and agentic workers, reuses valid computation, gathers evidence, recovers from failures and returns a verified state. Repository hosts remain systems of record for code; clouds remain providers of compute; WORKS owns execution truth.

22. Sources & Research Notes

Competitive facts in this report were checked against current vendor documentation on 31 August 2026. Vendor performance claims are treated as vendor claims until independently benchmarked.

GitHub Docs - Actions runner pricing

<https://docs.github.com/en/billing/reference/actions-runner-pricing>

Buildkite - Pricing and platform features

<https://www.buildkite.com/pricing/>

Nx - Orchestration & CI with Nx Cloud

<https://nx.dev/docs/features/ci-features>

Nx - Distribute Task Execution (Nx Agents)

<https://nx.dev/docs/features/ci-features/distribute-task-execution>

Nx - AI-Powered Self-Healing CI

<https://nx.dev/docs/features/ci-features/self-healing-ci>

Depot - GitHub Actions runners documentation

<https://depot.dev/docs/github-actions/overview>

Final recommendation

Proceed to founder validation and a Linux-first technical alpha. Do not start by implementing every platform feature in this report. The first proof must be brutally simple: a real repository migrates in minutes, executes reliably on WORKS, recovers from a worker failure, and shows a measurable time/cost advantage. Once that works, native Work and evidence can justify the larger category thesis.

WORKS should be built as a standalone venture from day one. AVC or any other internal system may become a design partner or customer, but must not define the product architecture.